LOAN DEFAULT DATA SCIENCE MIT FINAL PROJECT MILESTONE

Problem Definition

Why is this problem important to solve?

The manual loan underwriting process is often slow, prone to errors, and can overlook qualified applicants. Automating and streamlining this process with machine learning could significantly improve efficiency and accuracy, potentially approving more loans.

Objective

Automate and simplify the loan underwriting process for home equity lines of credit by building a classification model that predicts what clients are likely to default on their loan more accurately.

Apart from the core business problem. The manual loan underwriting process can lead to increased costs due to manual underwriting.

Key questions that need to be addressed:

What are the cons of using a manual underwriting process:

- Labor-intensive: Processing loans manually requires a dedicated team of underwriter, which can be expensive
- Operational Inefficiencies: Manual processes can be time-consuming, leading to delays in loan approvals and increased operational costs.

What is the main problem of using a manual underwriting process:

- Lost Opportunities: If qualified applicants are rejected due to errors or inefficiencies in the manual process, the business may miss out on potential revenue.

What is that we are trying to solve using data science?

Optimize loan underwriting through data-driven automation. Enhance prediction accuracy, Streamline the process, improve decision-Makin, Reduce risk

Data Exploration

The Home Equity dataset contains baseline and loan performance information for recent equity loans. Based on the data dictionary, this dataset originates from a financial institution such as mortgage lender with an emphasis to determine loan applications and their outcomes.

Loan Performance: The BAD variable is binary data type that indicates if loan was repaid or defaulted

Python Inputting libraries

```
import numpy as np
import pandas as pd

#import libraries for data visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#Inporting Libraries for Alogorithm Models
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
from google.colab import drive
drive.mount('/content/drive')
#Df = pd.read_csv('hmeq.csv')
Df= pd.read_csv('/content/drive/MyDrive/Colab Notebooks/hmeq.csv')
#returns first 5 rows
Df.head()
```

Data Dictionary:

Shape of the data:

(5960 rows,13 columns)

Data Field	Definition	Data type
BAD	1 = Client defaulted on loan, 0 = loan repaid	Int64
LOAN	Amount of loan approved	Int64
MORTDUE	Amount due on the existing mortgage	Float64
VALUE	Current value of the property	Float64
REASON	Reasonfortheloanrequest(HomeImp=homeimprovement,DebtCon=debtconsolidationwhichmeanstakingoutanewloantopayoffotherliabilitiesandconsumerdebts	Object
JOB	The type of job that loan applicant has suchas manager, self, etc	Object
YOJ	Years at present job	Float64
DEROG	Numberofmajorderogatoryreports	Float64
DELINQ	Numberofdelinquentcreditlines	Float64
CLAGE	Age of the oldest credit line in months	Float64
NINQ	Number of recent credit inquiries	Float64
CLNO	Number of existing credit lines	Float64
DEBTINC	Debt-to-incomeratio	Float64

Missing Values:

```
Df.isnull().sum()
```

Data Field	Missing Values
BAD	0
LOAN	0
MORTDUE	518
VALUE	112
REASON	252
JOB	279
YOJ	515
DEROG	708
DELINQ	580
CLAGE	308
NINQ	510
CLNO	222
DEBTINC	1267

Statistical Summary:

```
Summary=Df.describe()
print(Summary)
```

```
                BAD           LOAN        MORTDUE          VALUE          YOJ
count   5960.000000   5960.000000    5442.000000    5848.000000  5445.000000
mean       0.199497  18607.969799   73760.817200  101776.048741     8.922268
std        0.399656  11207.480417   44457.609458   57385.775334     7.573982
min        0.000000   1100.000000    2063.000000    8000.000000     0.000000
25%        0.000000  11100.000000   46276.000000   66075.500000     3.000000
50%        0.000000  16300.000000   65019.000000   89235.500000     7.000000
75%        0.000000  23300.000000   91488.000000  119824.250000    13.000000
max        1.000000  89900.000000  399550.000000  855909.000000    41.000000

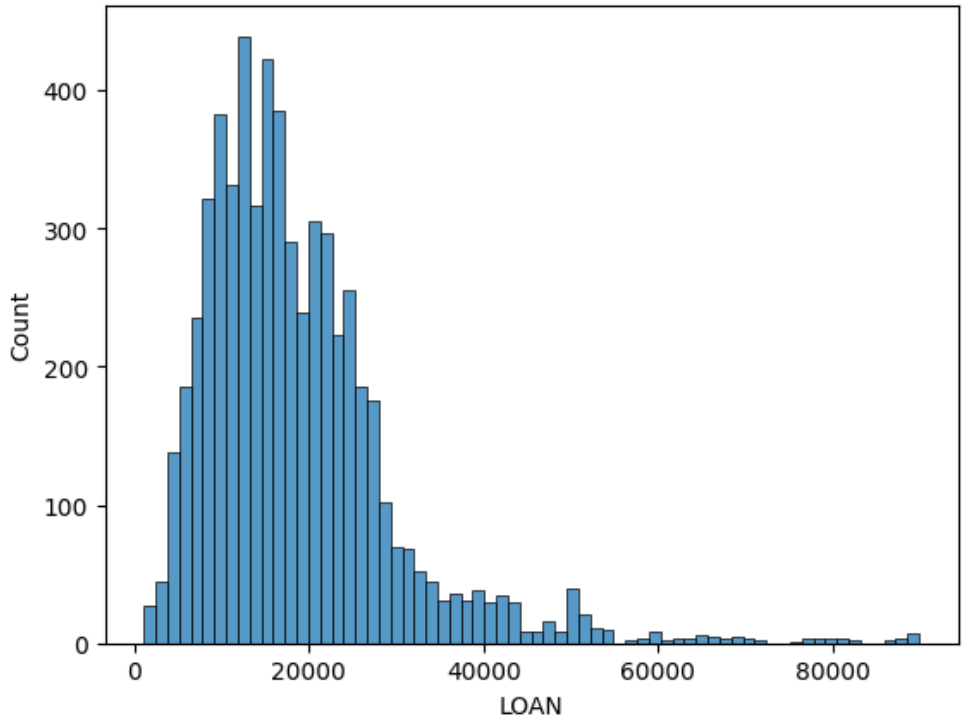
             DEROG       DELINQ         CLAGE          NINQ          CLNO  \
count  5252.000000  5380.000000  5652.000000  5450.000000  5738.000000
mean      0.254570     0.449442   179.766275     1.186055    21.296096
std       0.846047     1.127266    85.810092     1.728675    10.138933
min       0.000000     0.000000     0.000000     0.000000     0.000000
25%       0.000000     0.000000   115.116702     0.000000    15.000000
50%       0.000000     0.000000   173.466667     1.000000    20.000000
75%       0.000000     0.000000   231.562278     2.000000    26.000000
max      10.000000    15.000000  1168.233561    17.000000    71.000000
```

```
            DEBTINC
count   4693.000000
mean      33.779915
std        8.601746
min        0.524499
25%       29.140031
50%       34.818262
75%       39.003141
max      203.312149
```

```
#histogram for cost of order
sns.histplot(data=Df,x='LOAN')
```

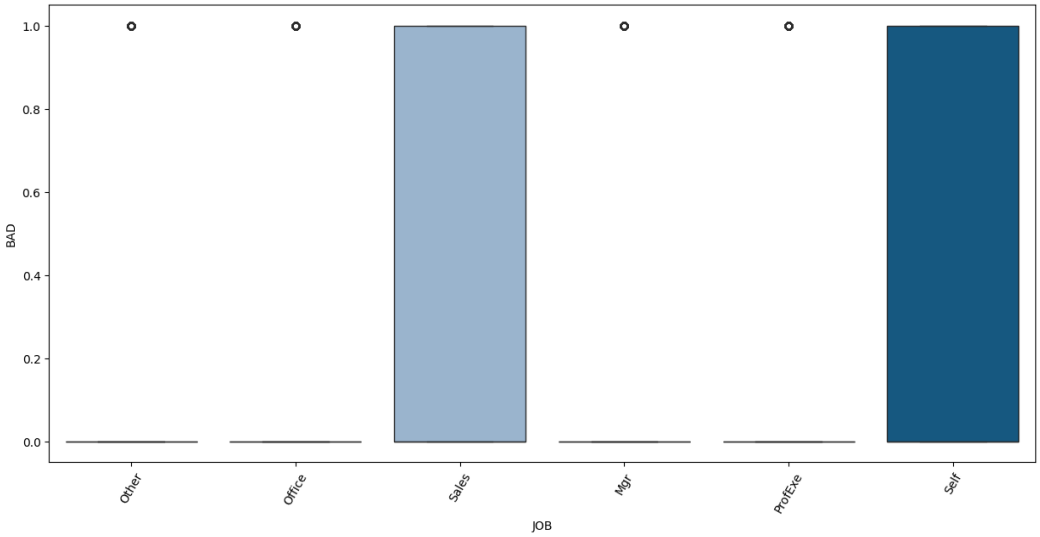
Observations about the data:

Histogram of LOAN (Amount of Loan approved): Based on this histogram extracted from Google collabs. The majority of loans ~ 75% approved lie on the 0-20,000 Bin , and ~25% between 20,000 to 40,000.



Identify what job category type defaulted loans more than others?

```
plt.figure(figsize=(15,7))
sns.boxplot(x= "JOB", y="BAD", data= Df, palette= 'PuBu' )
plt.xticks(rotation=60)
plt.show()
```



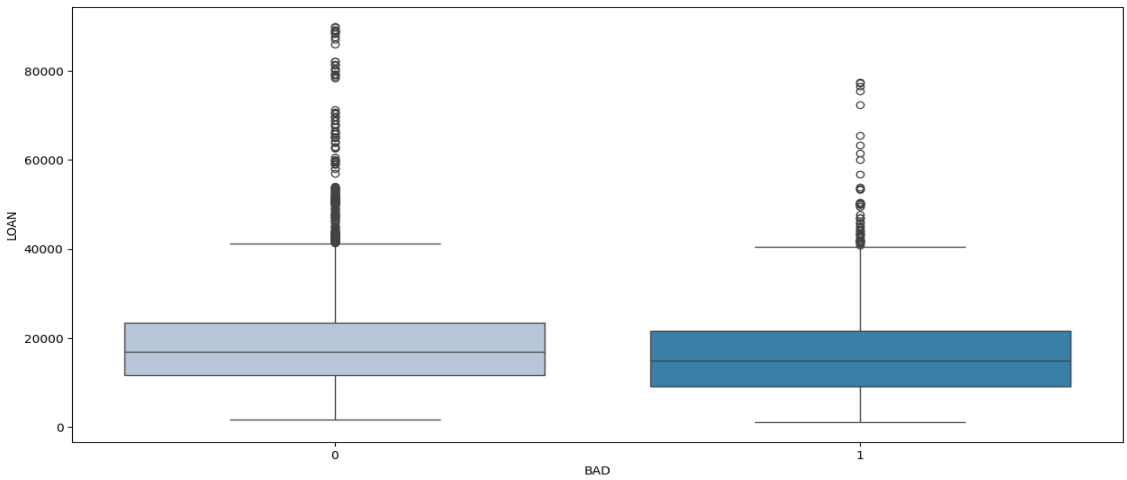
What do these observations mean for the Problem Formulation?

Based on the image extracted from Google collabs, Sales job category type and Self job category were the categories that defaulted on paying back the loans. While, Office, Mgr , Prof Exc were able to pay back their loans. Based on this, we can infer that these jobs (Sales, Self) have a high probability of defaulting on the loan.

Outliers:

60,000 to 80,000 of loans approved

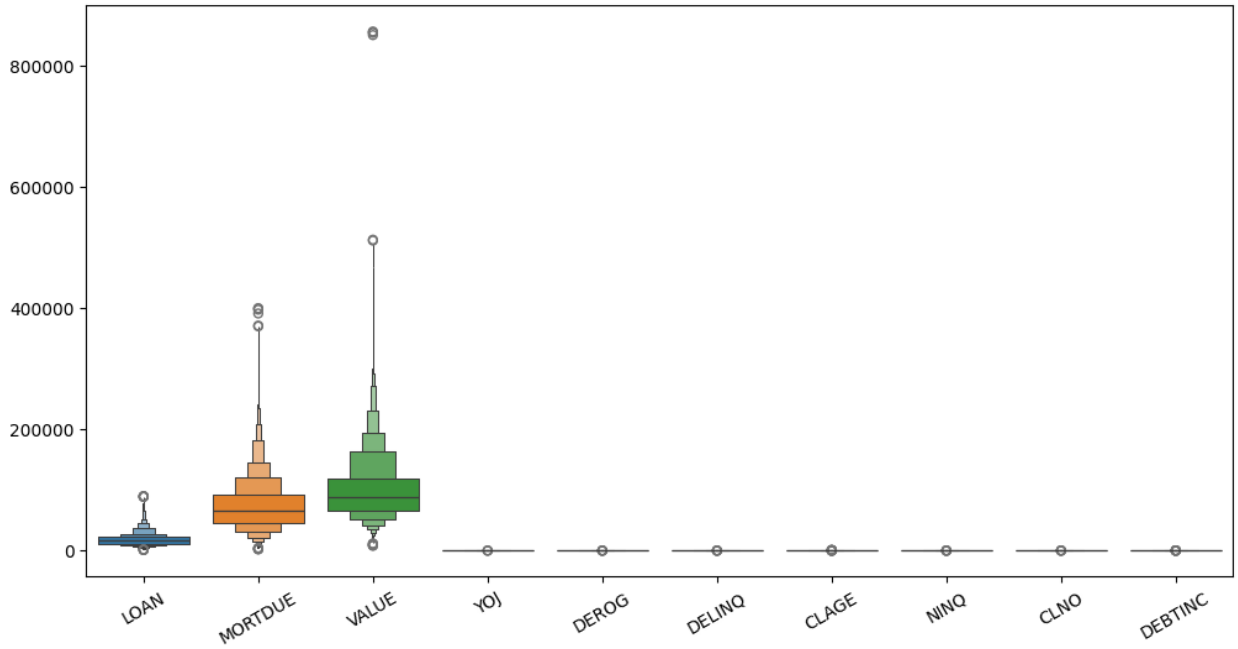
```
plt.figure(figsize=(15,7))
sns.boxplot(x = "BAD", y= "LOAN", data= Df, palette= 'PuBu')
plt.xticks(rotation=0)
plt.show()
```



As we can see in the picture above, we can observe that the percentage of approved loans vs non-approved loans are almost equally distributed. And the outliers are also distributed from 40,000 and above.

Outliers

```
#range of data for  Ourliers
plt.figure(figsize = (12,6))
sns.boxenplot(data = df.drop(columns = 'BAD'))
plt.xticks(rotation = 30)
plt.show()
```



Based on the chart above, we can see the outliers. MORTDUE , there are data between 20,000 to 400,000 and VALUE from ~ 25,000 to 450,000 which can skew the model significantly. To decrease this skewness ,we will drop one variable from the model .

VALUE has significant outliers. “Value” which is current value of the property does not seem to serve us well to predict the model; hence. We will drop this variable from the logistical regression

```
df.loc[df['VALUE']==df['VALUE'].max()]
```

	BAD	LOAN	MORTDUE	VALUE	YOJ	DEROG	DELINQ	CLAGE	NINQ	CLNO	DEBTINC
5752	1	43500	NaN	855909.0	16.0	0.0	0.0	196.403636	0.0	23.0	42.985408

```
df.drop(5752, axis = 0, inplace = True)
#Scaling the data
X = df.drop(columns = 'BAD')
```

```
y = df['BAD']

X_std = StandardScaler().fit_transform(X)
X_std = pd.DataFrame(X_std, columns = list(X.columns))
```

Building Models

Building various models and tuning the model to get better validation- What different techniques should be explored?

Project Loan Default offers a very important variable called "BAD" which can be used as the predictor of how many loans are likely to default. For this project, we will explore a Logistical Regression Model which will enable us to predict the percentage of loans likely to pay and default. We will determine the significance of all these independent variables and determine their results:

Feature Importance:

```
# trained Logistic Regression model
coefficients = model.coef_[0]  # Get the coefficients for the first class
(default)

# Create a DataFrame to display feature names and coefficients
feature_importance = pd.DataFrame({'Feature': X_train.columns,
'Coefficient': coefficients})

# Sort absolute coefficient value to see the most significant features
feature_importance = feature_importance.sort_values(by=['Coefficient'],
key=abs, ascending=False)

print(feature_importanc
```

Results:

Feature	Coefficient
DEBTINC	.88
DELINQ	.61
DEROG	.45
CLAGE	-.39
CLNO	-.021
NINQ	0.16
LOAN	-0.16
VALUE	0.13
YOJ	-0.07
MORTDUE	0.0009

The feature with the highest significance is DEBTINC because it has the highest absolute coefficient

Logistical Regression

Python Code

Inputting Data and Splitting data

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix, roc_auc_score, roc_curve, log_loss
import numpy as np
from sklearn.impute import SimpleImputer

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
stratify=y, random_state=14)
# Create an imputer to replace NaNs with the mean of each column
imputer = SimpleImputer(strategy='mean')
# Handle missing values and rescale
X_train = X_train.dropna()
y_train = y_train[X_train.index]

# Fit the imputer on your training data (after dropping NaNs)
imputer.fit(X_train)

# Transform both training and test data using the imputer
X_train_imputed = imputer.transform(X_train)
X_test_imputed = imputer.transform(X_test)
# Rescale data after dropping missing values:
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_imputed)
X_test_scaled = scaler.transform(X_test_imputed)
# Create and fit the model with adjusted parameters
model = LogisticRegression(solver='saga', penalty='l2', C=1.0,
max_iter=1000)  # Adjusted solver, penalty, and C
model.fit(X_train_scaled, y_train)
# Fit using scaled data
```

LogisticRegression?i

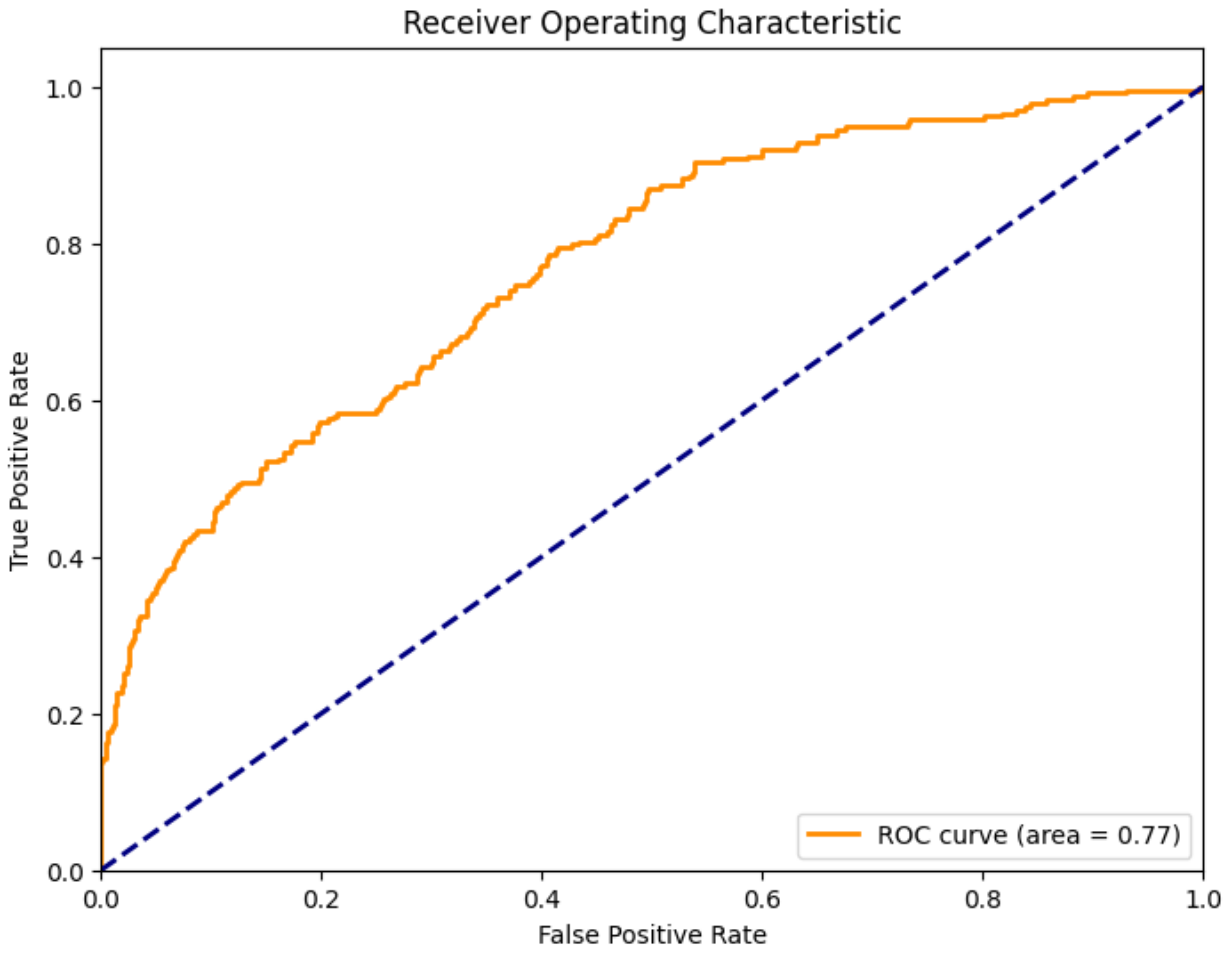
```
LogisticRegression(max_iter=1000, solver='saga')
```

```
# Assuming you have your model predictions:
y_pred = model.predict(X_test_scaled)
```

```
y_pred_prob = model.predict_proba(X_test_scaled)[:, 1]
# Calculate metrics:
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_prob)
logloss = log_loss(y_test, y_pred_prob)
# Print or visualize the results
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1-Score: {f1}")
print("Confusion Matrix:\n", conf_matrix)
print(f"ROC AUC: {roc_auc}")
print(f"Log Loss: {logloss}")
```

Logistic Regression Model Score

Accuracy	83.22%
Precision	78.9%
Recall	0.218
F1 -Score	0.342
ROC AUC	0.778
Log Loss	0.443



Overall Performance:

The model predicts about 83.22% accuracy to be correct. With an ROC AUC of 0.78 suggesting that the model is better than random distinguishing between default and no default.

Class-Specific Performance

Of all the clients predicted to default, 78.79% defaulted , this is important to minimize false positives. Furthermore, the model identified 21.8% of actual defaulters. This can become problematic when identifying high-risk clients.

Finally, the F1-Score of 0.34 seems to be low and we will attempt to enhance this score by using the same variables to build a random forest classifier:

Proposal for the final solution design

- Is the model performance good enough for deployment in production and is it interpretable? - What model do you propose to be adopted? - Why is this the best solution to adopt? - How that solves the problem?

We have used the feature importance to determine which variable has the highest significance when determining who is likely to pay the loan and who is not. The Model that I propose to be adopted is The

enhance Logistic regression with a random forest feature reengineering so we can single pick the most important feature of the data and advice the company on what to focus.

Improve the Model

Proposal for the final solution design

- How these techniques can be improved? - Why the techniques suggested are the best to explore for the data and problem at hand?

After observing a relatively low F1 score, of 0.34, we will aim to improve this model by selecting the most important feature and adding to the model. “DEBTINC” Debt to Income

We will build a random forest classifier by adding a new feature

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

# Make predictions on the test set
y_pred = rf_model.predict(X_test_scaled)

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

# Print the results
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Confusion Matrix:\n", conf_matrix)
```

Accuracy: 84%
Precision: 78.3%
Recall: .39
F1 Score: .45
Confusion Matrix:

We've managed to improve the model with an enhanced F1 score of .45 . This is better and states that the Debt to Income explains about 45% of the probability that a borrower will default or not default on a loan.

Confusion Matrix

933	21
162	76

True Negatives : 933 clients were correctly predicted as not defaulting
False Positives : 21 clients were incorrectly presicted as defaulting Type I error
False Negatives: 162 clients were incorrectly predicted as not defaulting Type II error
Ture Positives : 76 clients were correctly predicted as defaulting.